Smart Library & Fine Management System

Software Engineering Sessional Project Report

Student Name: **[PLACEHOLDER: your name]**
Student ID: **[PLACEHOLDER: your ID]**
Supervisor/Instructor: **[PLACEHOLDER: instructor name]**
Department of Computer Science & Engineering
Rajshahi University of Engineering & Technology (RUET)

[PLACEHOLDER: submission date]

Abstract

This report presents the design, implementation and verification of the Smart Library & Fine Management System, a full-stack web application developed for a small academic library. The system follows the complete Software Development Life Cycle using the Incremental Model and is built on the MERN stack (MongoDB, Express, React, Node.js) with Tailwind CSS. Five functional modules are implemented: member management, book catalog, borrow/return with borrowing limits, overdue fine management, and audit with in-application notifications. Three design patterns from three categories are applied in production code: Singleton (shared database manager), Facade (borrow/return workflow) and Strategy (interchangeable fine policies). Verification combines Test-Driven Development with Jest, dependency mocking, Supertest integration tests against an isolated in-memory MongoDB (64 tests, 89.8% statement coverage at the time of writing), a Selenium WebDriver end-to-end test of the issue/return workflow, and a GitHub Actions continuous-integration pipeline. Swagger/OpenAPI documentation, learned independently, provides interactive API documentation at `/api-docs`. The report documents honest limitations, including the absence of authentication and of multi-document MongoDB transactions.

Keywords: MERN stack, software engineering, design patterns, Test-Driven Development, Selenium, continuous integration, Swagger/OpenAPI, fine calculation.

Contents

1 Introduction **3**

2 Problem Statement **3**

3 Objectives and Scope **3**

4 Functional Requirements **4**

5 Non-Functional Requirements **5**

6 User Stories and Use Cases **5**

7 SDLC Process Model **5**

8 System Architecture **5**

9 Module Descriptions **6**

10 UI Prototype **6**

11 MongoDB Data Model **6**

12 REST API **8**

13 Design Patterns **8**

13.1 Singleton — DatabaseManager 8

13.2 Facade — BorrowFacade 9

13.3 Strategy — Fine Calculation 9

14 UML Diagrams **10**

15 Implementation Overview **12**

16 TDD Workflow **13**

17 Mocking and Integration Testing **13**

18 Selenium Testing **14**

19 Git/GitHub Configuration Management **14**

20 Pull-Request Review Evidence **14**

21 Merge-Conflict Resolution Evidence **14**

22 Continuous Integration (GitHub Actions) **15**

23 Swagger/OpenAPI — Self-Directed Tool **15**

24 Learning Log **15**

25 Reflection Essay (500–700 words) **16**

26 Limitations **17**

27 Future Improvements **18**

28 Conclusion **18**

29 References **18**

A Key Code Excerpts **19**

B Test Results **19**

B.1 Coverage Summary 20

B.2 TDD Red–Green Record 20

B.3 Selenium E2E 20

C Git Evidence **21**

D Pull-Request Evidence **21**

E Merge-Conflict Evidence **21**

F CI Evidence **21**

G Swagger Evidence **21**

H Learning Log **21**

I Application Screenshots **21**

List of Figures

1 Incremental development model used in this project (source: `docs/diagrams/sdlc-increment`

2 Three-tier architecture with pattern locations 6

3 UI wireframe (planning prototype) 6

4 Document relationships (logical references, not relational FKs) 7

5 UML class diagram (actual implemented classes) 10

6 Sequence diagram: issuing a book via BorrowFacade 11

7 Sequence diagram: return with Strategy-based fine calculation 12

8 Use-case diagram (actors and included sub-functions) 12

1 Introduction

Libraries in academic institutions still manage borrowing records with spreadsheets or manual registers, which are error-prone when calculating overdue fines and tracking availability. This project builds a small but complete web-based library information system that manages members, books, borrowing and returning, overdue fines and an audit trail.

The project serves a dual purpose: (1) a practical product for a small library, and (2) a vehicle to demonstrate Software Engineering techniques required by CSE 3206: requirements engineering, process models, design patterns, Test-Driven Development, mocking, integration testing, Selenium, configuration management, continuous integration and an independently learned tool (Swagger/OpenAPI).

Note on the assignment sheet: the original specification contains an inconsistency — the heading states 10 marks while the marking scheme lists 20 marks. The student should confirm the official mark distribution with the course instructor; this report does not assume either value.

2 Problem Statement

A small academic library must handle a growing set of members (students, faculty and staff), a book catalog with limited copies, and an overdue fine policy that differs by member category. Manual tracking leads to: incorrect availability counts, forgotten due dates, inconsistent fine calculations and no traceability of who did what. The problem is to design and implement a maintainable, testable and extensible system that automates these five concerns with persistent storage.

3 Objectives and Scope

Objectives

1. Implement five functional modules with persistent MongoDB storage.
2. Apply at least three design patterns from three categories in real workflows.
3. Practice TDD, mocking and integration testing with measurable coverage.
4. Use Git feature branches, pull requests and CI for configuration management.
5. Document the API with Swagger/OpenAPI (independently learned tool).
6. Produce a complete engineering report with UML and test evidence.

In scope: role-labelled members (no authentication), issue/return with limits, fine calculation, audit log, web UI, API documentation, CI.

Out of scope: authentication/authorization, online payments, email or SMS notifications, barcode/RFID, multi-branch libraries, pagination.

4 Functional Requirements

The numbered functional requirements (FR-1.1 to FR-5.4) across the five modules are listed in Table 1 and fully specified in `docs/requirements.md`. Each FR is covered by at least one automated test.

Table 1: Functional requirement summary

Module	Requirements
Members	FR-1.1 register with unique ID; FR-1.2 reject duplicates (409); FR-1.3 validate fields/role; FR-1.4 search and filter.
Books	FR-2.1 add with unique ISBN and copies; FR-2.2 reject duplicates and invalid counts; FR-2.3 search by title/author/ISBN; FR-2.4 $0 \leq$ available $\leq$ total.
Borrow/return	FR-3.1 issue with $+14$-day due date; FR-3.2 limits (Student 3, Faculty 5, Librarian 0); FR-3.3 validate member/book/stock; FR-3.4 return restores stock (capped); FR-3.5 list active/returned.
Fines	FR-4.1 whole overdue days, never negative; FR-4.2 Student 10 BDT/day; FR-4.3 20 BDT/day after day 7; FR-4.4 Faculty 0 BDT; FR-4.5 on-time return = 0; FR-4.6 overdue list with live fines.
Audit	FR-5.1 record issue/return/fine/overdue/member/book events; FR-5.2 typed events with message and timestamp; FR-5.3 success/error messages surfaced in the UI; FR-5.4 filterable history.

Fine policy boundary conditions (FR-4.2/4.3): let d be whole overdue days. For students,

$$\text{fine}(d) = \begin{cases} 10d & 0 \leq d \leq 7 \\ 70 + 20(d-7) & d > 7 \end{cases}$$

so $d = 7 \Rightarrow 70$ BDT and $d = 8 \Rightarrow 90$ BDT. Faculty: $\text{fine}(d) = 0 \ \forall d$. A book returned on or before the due date has $d = 0$ and fine 0.

5 Non-Functional Requirements

Eight non-functional requirements cover usability (one screen per module, visible feedback), maintainability (small single-responsibility modules), data integrity (unique IDs, atomic stock updates, non-negative fines), reliability (centralised error handler, consistent JSON errors), basic security (no credentials in code, input validation), performance (sub-second responses for a small library), testability (DI + mocks + isolated test DB) and extensibility (new fine policy = one new class). Full text: `docs/requirements.md` §2.

6 User Stories and Use Cases

Nine user stories (e.g. "As a librarian, I want to process a return that automatically calculates any fine, so that I do not have to count overdue days by hand") and three fully written use cases (Issue a Book; Return a Book and Calculate Fine; View Overdue Fines) with actors, preconditions, main and alternative flows are documented in `docs/requirements.md` §3–4.

7 SDLC Process Model

The **Incremental Model** was selected (Figure 1). The five modules are largely independent; each increment produces a working, testable slice: (I1) requirements and design, (I2) members and books, (I3) borrow/return, (I4) fines and audit, (I5) testing hardening, Swagger, CI, Selenium and documentation. Incremental development matches a single-developer sessional project: risks are isolated per increment, and a defect in the fine policy cannot destabilise member registration because they are separate increments and separate packages.

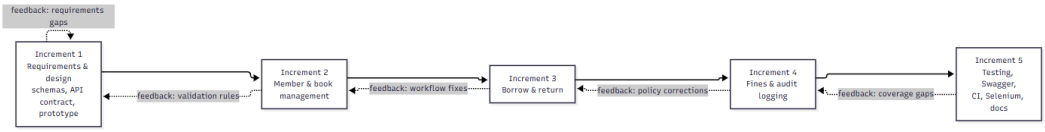


Figure 1: Incremental development model used in this project (source: `docs/diagrams/sdlc-incremental.mmd`)

8 System Architecture

Three tiers: a React single-page application talks JSON/HTTP to an Express REST API, which persists through Mongoose to MongoDB. Inside the server, controllers are thin; the **BorrowFacade** coordinates multi-step workflows; repositories isolate persistence;

the **FineService** selects a **Strategy** for each fine policy; **DatabaseManager** (Singleton) owns the single connection manager. Figure 2 shows the component view.

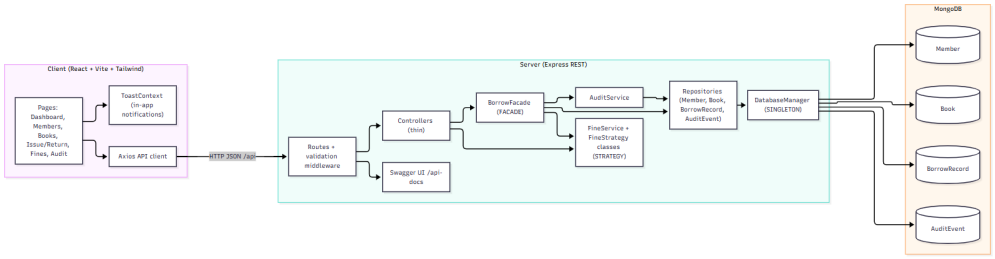


Figure 2: Three-tier architecture with pattern locations

9 Module Descriptions

M1 Members: unique `memberId`, roles Student/Faculty/Librarian, search/filter; implemented in `memberController.js` + `MemberRepository` + React page `Members.jsx`. **M2 Books:** unique ISBN, copy counters guarded by schema validation and an atomic conditional update; `bookController.js`, `Books.jsx`. **M3 Borrow/return:** `BorrowFacade.issueBook/` 14-day loans; per-role limits; stock updates; `BorrowReturn.jsx`. **M4 Fines:** strategy-based calculation, live overdue endpoint; `Fines.jsx`. **M5 Audit/notifications:** `AuditService` persists typed events; `ToastContext.jsx` shows in-app success/error messages; `Audit.jsx` renders the history.

10 UI Prototype

A low-fidelity wireframe was produced before implementation (`docs/diagrams/ui-wireframe.md`, Figure 3). It planned the sidebar layout, dashboard cards, member/book forms and tables, the issue/return workflow, the fines policy cards and the audit list. The prototype is a planning artefact and is not presented as evidence of functioning software.

[PLACEHOLDER: optional: render the wireframe sketch to figures/ui-wireframe.png]

Figure 3: UI wireframe (planning prototype)

11 MongoDB Data Model

Four collections: `Member`, `Book`, `BorrowRecord` and `AuditEvent` (Table 2). MongoDB is document-oriented: there are no foreign keys. `BorrowRecord` stores the `_id` of the member and book as *logical references*, resolved in the API with Mongoose

`populate()` when names are needed for the UI. Indexes cover unique `memberId`/ISBN, text search on title/author, and the overdue query (`status, dueDate`).

Table 2: MongoDB collections (Mongoose schemas)

Collection	Fields
Member	_id, memberId (unique), name, email, role (Student/Faculty/Librarian), createdAt
Book	_id, isbn (unique), title, author, totalCopies (≥ 1), availableCopies ($0 \ldots$totalCopies), timestamps
BorrowRecord	_id, memberId $\rightarrow$ Member, bookId $\rightarrow$ Book, borrowedAt, dueDate, returnedAt, status (active/returned), fineAmount (≥ 0)
AuditEvent	_id, eventType (enum of 6 types), memberId?, borrowRecordId?, message, createdAt (index, descending)

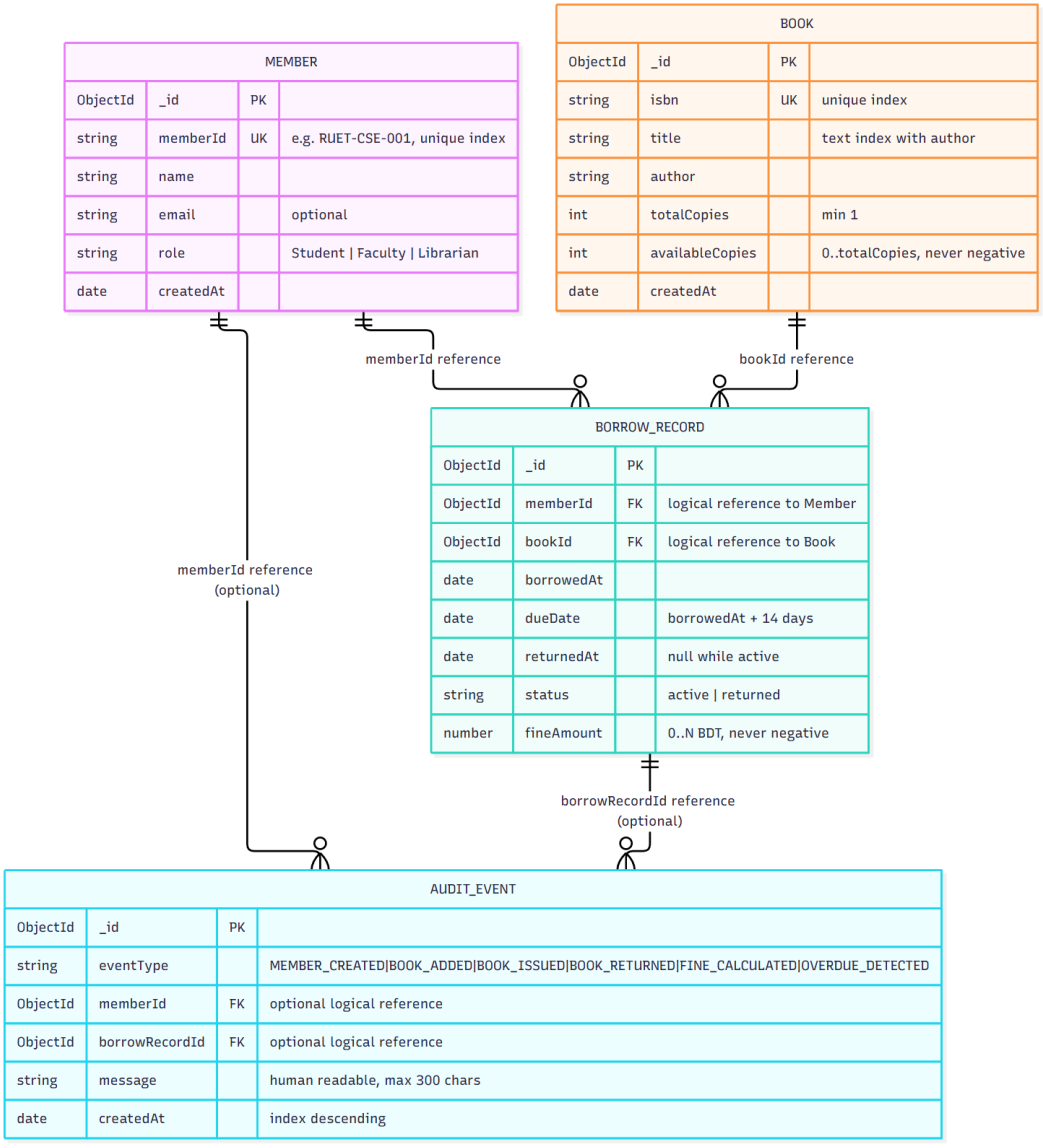


Figure 4: Document relationships (logical references, not relational FKs)

12 REST API

Thirteen endpoints (Table 3) are implemented, validated and documented in the OpenAPI file `server/docs/openapi.yaml`. Errors use a consistent body `{"success": false, "message"` via one centralised error handler; status codes: 400 validation, 403 role, 404 not found, 409 conflict, 500 unexpected.

Table 3: REST API endpoints

Method	URL	Purpose
POST	/api/members	Register member (409 on duplicate ID)
GET	/api/members?q=&role=	List/search members
GET	/api/members/:id	Member details
POST	/api/books	Add book (409 on duplicate ISBN)
GET	/api/books?q=	Search by title/author/ISBN
GET	/api/books/:id	Book details
POST	/api/borrowings/issue	Issue {memberId, bookId} (Facade)
POST	/api/borrowings/return/:id	Return + fine calculation
GET	/api/borrowings?status=	Active/returned records
GET	/api/fines/overdue	Overdue list with live fines
GET	/api/fines/preview/:id	Fine preview (read-only)
GET	/api/audits?eventType=	Audit history
GET	/api/stats	Dashboard statistics

13 Design Patterns

13.1 Singleton — DatabaseManager

Category: creational. Problem: the application must not create a new MongoDB connection manager per request; initialisation and errors must be handled in one place. **Location:** `server/src/database/DatabaseManager.js`. `getInstance()` returns the single instance held in a static field; `connect()` reuses an existing connection. **Benefit:** one owner of the connection lifecycle, predictable shutdown in tests. **Honest note:** the MongoDB driver already pools connections — Singleton does not mean one socket and does not automatically improve throughput. **Rejected alternative:** Prototype — cloning a database manager is meaningless here; there is no state worth duplicating and duplicated managers would fragment the connection lifecycle.

Listing 1: Singleton core (excerpt)

```
class DatabaseManager {
  static instance = null;
  static getInstance() {
    if (DatabaseManager.instance === null) {
      DatabaseManager.instance = new DatabaseManager();
    }
```

```
    return DatabaseManager.instance;
  }
  async connect(uri) { /* reuse or establish connection */ }
}
```

13.2 Facade — BorrowFacade

Category: structural. Problem: issuing a book touches member lookup, role/limit checks, atomic stock decrement, record creation, due-date assignment and audit logging; returning adds fine calculation. Without a facade, controllers would duplicate and mix this workflow. **Location:** `server/src/facades/BorrowFacade.js`. The controller calls exactly `borrowFacade.issueBook(memberId, bookId)` and `returnBook(id)`. Persistence and fine calculation are delegated to injected repositories and services (kept small on purpose). **Benefit:** thin controllers, one readable workflow, unit-testable with mocks. **Rejected alternative:** Adapter — Adapter solves *interface incompatibility* with a third-party API; here all collaborators are our own classes and the need is *orchestration*, not interface conversion.

13.3 Strategy — Fine Calculation

Category: behavioural. Problem: fine rules differ by member category and are expected to change; burying `if role == ...` in a controller would scatter the policy. **Location:** `server/src/strategies/` — `StudentFineStrategy` (10 BDT/day), `FacultyFineStrategy` (0), `OverdueHeavyFineStrategy` (decorator: first 7 days at the base rate, then 20 BDT/day). `FineService.getStrategyForRole()` selects and composes them behind the shared `calculateFine(days)` interface. **Benefit:** substitutable policies; a new policy (e.g. Staff) is a new class plus one switch case — existing strategies remain untouched (open/closed). **Rejected alternative:** State — State models one object whose behaviour changes with its *internal state*; here we have a family of interchangeable algorithms chosen by an *external* attribute (role), which is Strategy's definition.

Listing 2: Strategy selection (excerpt)

```
getStrategyForRole(role) {
  switch (role) {
    case 'Student':
      return new OverdueHeavyFineStrategy(
        new StudentFineStrategy(10), 7, 20);
    case 'Faculty':
    case 'Librarian':
      return new FacultyFineStrategy();
    default:
      throw new Error(`Unknown member role: ${role}`);
  }
```

```
}
```

Test evidence: `tests/unit/fineStrategy.test.js` verifies 0/1/5/7/8/10/14-day boundaries and invalid inputs; `tests/unit/borrowFacade.test.js` verifies facade coordination with fully mocked dependencies.

14 UML Diagrams

Figure 5 shows the class diagram containing the actual classes of the codebase; Figures 6 and 7 show the issue and return+fine sequences. Mermaid sources are in `docs/diagrams/`.

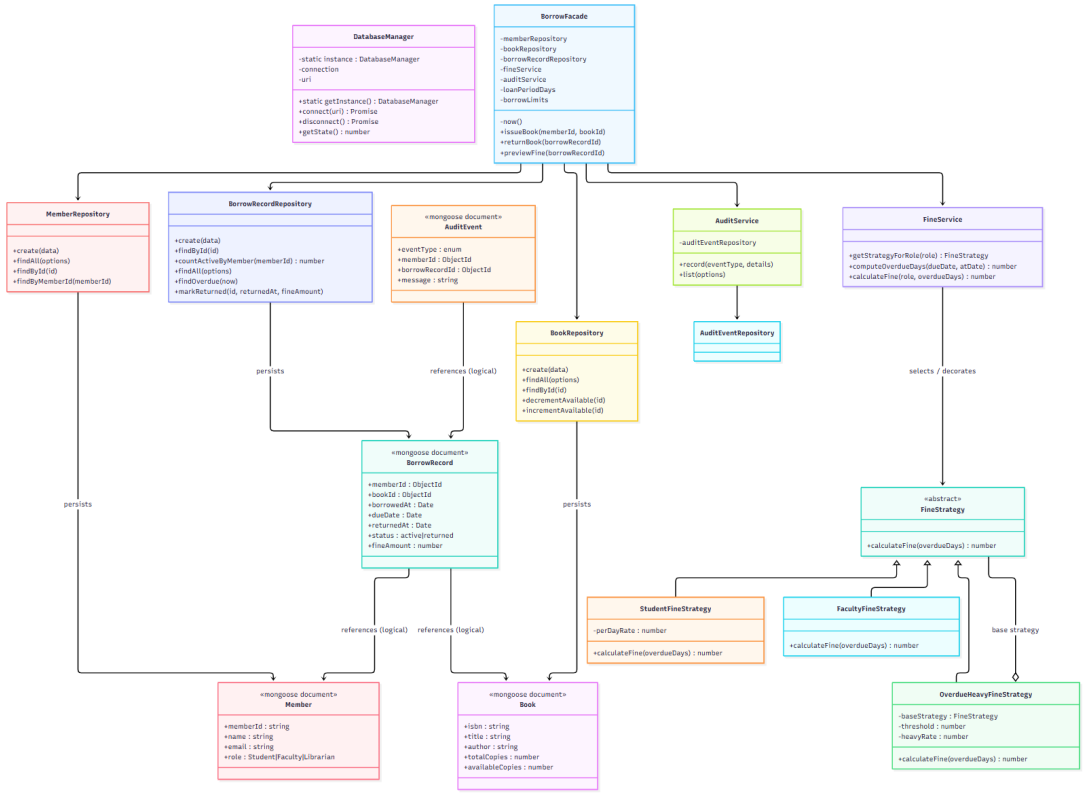


Figure 5: UML class diagram (actual implemented classes)

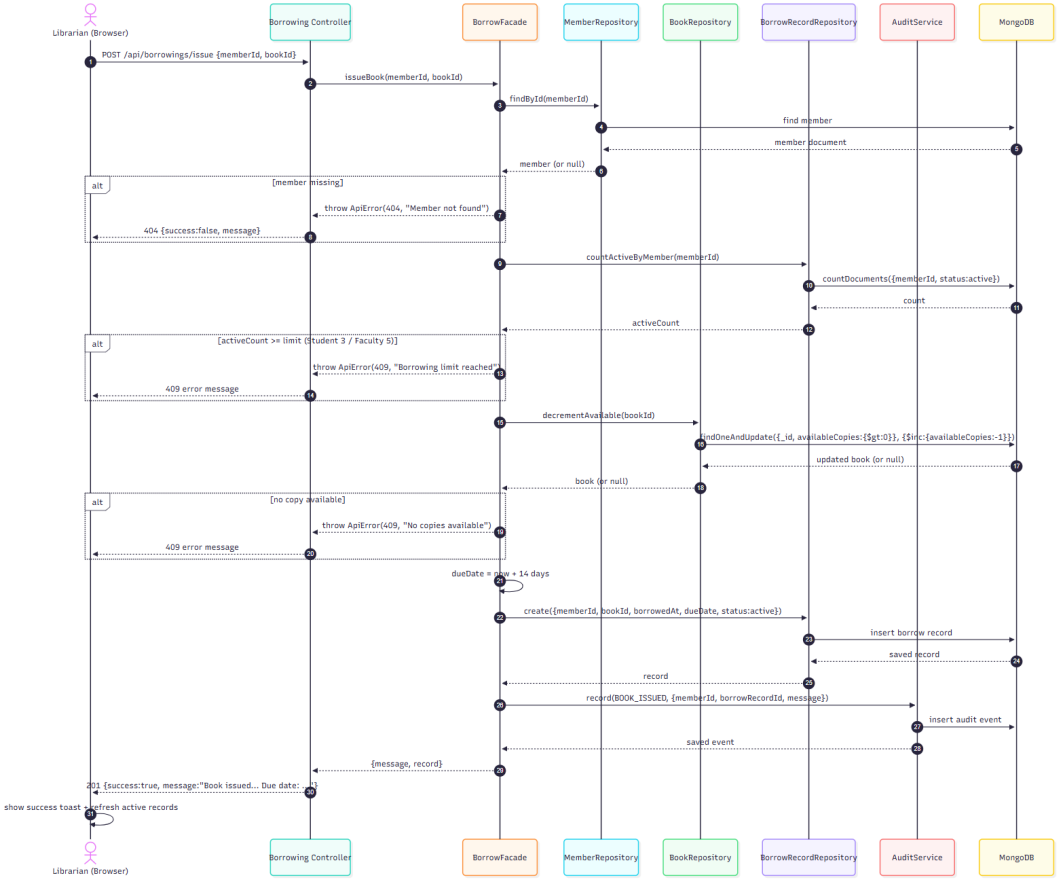


Figure 6: Sequence diagram: issuing a book via BorrowFacade

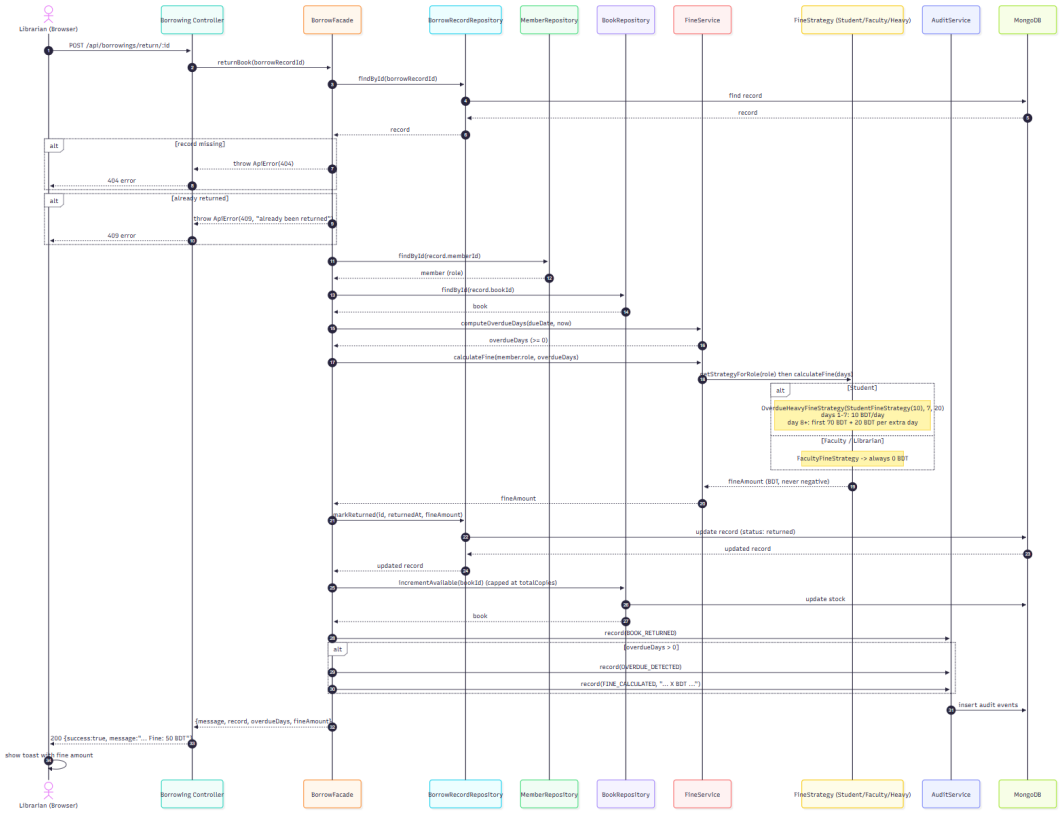


Figure 7: Sequence diagram: return with Strategy-based fine calculation

A use-case diagram is shown in Figure 8; sources: `docs/diagrams/use-case.mmd` (UML notation) and `use-case-graph.mmd` (renderable variant).

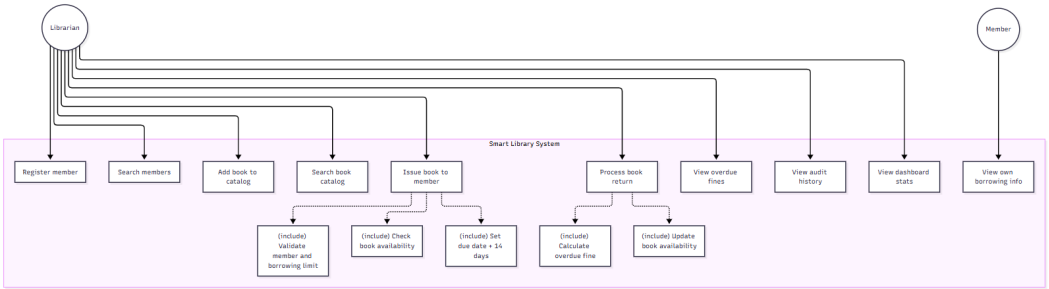


Figure 8: Use-case diagram (actors and included sub-functions)

15 Implementation Overview

Backend. Node.js 22 + Express 4. `app.js` performs manual dependency injection: repositories → services → facade → controllers → routes, so tests can replace any collaborator. Request validation uses a small custom middleware (`validate.js`); errors flow to `errorHandler.js`. Stock safety: `decrementAvailable` uses a conditional atomic update `{availableCopies: {$gt: 0}} with $inc: -1`, so two concurrent issues cannot both take the last copy. Multi-document transactions are *not* used

(a local standalone MongoDB does not support them without a replica set); the ordered facade workflow plus the atomic decrement is the documented, honest alternative — it is safe at this scale but is not a transactional guarantee.

Frontend. React 18 + Vite 5 + Tailwind CSS 3.4, React Router pages (Dashboard, Members, Books, Issue/Return, Fines, Audit), Axios client, and a `ToastContext` for in-app success/error notifications. Pages show loading, empty and error states and validate required fields before submitting.

16 TDD Workflow

Fine calculation was developed test-first. The recorded cycle (`docs/tdd-workflow.md`, logs in `docs/evidence/`): with the implementation in its unimplemented state, the Student strategy tests failed (`expected 50, received 0`; 2 failed / 3 passed) — **Red**; after implementing `return days * perDayRate`, all 5 focused tests passed — **Green**; extraction of the heavy-overdue decorator and service wiring with the suite still green — **Refactor**. TDD was chosen because fine rules are precise business logic whose silent breakage is expensive; boundary tests (day 7 vs. day 8) force the edge cases to exist.

17 Mocking and Integration Testing

Mocking. `tests/unit/borrowFacade.test.js` replaces `memberRepository`, `bookRepository`, `borrowRecordRepository`, `fineService` and `auditService` with `jest.fn()` objects. Assertions verify, for example, that exactly one `BOOK_ISSUED` audit event is recorded on success and *none* on failure paths — without any database.

Integration. `tests/integration/api.test.js` (28 tests) drives the real Express app with Supertest against an isolated MongoDB produced by `mongodb-memory-server` (or `MONGODB_URI_TEST` if provided), with full cleanup between tests — development and Atlas data cannot be touched. Workflows covered: member and book creation, duplicates, validation failures, issue, unavailable book, borrow limit, on-time return (fine 0), overdue returns (50 BDT at 5 days; 90 BDT at day 8 for students; 0 for faculty), fine endpoints, audits and dashboard stats.

Coverage (actual run, 64 tests, all passing): statements 89.77%, branches 73.33%, functions 91.01%, lines 90.88% (`npm run test:coverage`; full table in Appendix B.1). Branch coverage is lowest in the centralised error handler's rarely-hit branches — identified as an improvement target. Coverage measures execution, not correctness; it is reported together with the assertions' intent.

18 Selenium Testing

`e2e/tests/issueAndReturn.e2e.js` drives Chrome through the built React application: open dashboard → add book → register member → issue the book via the UI selects → assert the success toast ("Due date") and the active borrowing row → click Return → assert "No fine". Stable `data-testid` attributes are used as selectors. The orchestrator `e2e/run-e2e.js` starts the backend in in-memory demo mode, serves the frontend with `vite preview` on port 4173, waits for health, runs the test and shuts down. **Recorded result on the development machine: E2E TEST PASSED (6/6 steps)**; output in Appendix B.3. Selenium runs locally and in the CI job for ubuntu-latest (Chrome preinstalled, Selenium Manager fetches ChromeDriver).

19 Git/GitHub Configuration Management

Branch naming convention and full command recipes are in `docs/git-workflow.md`: `feature/member-management`, `feature/book-catalog`, `feature/borrow-return`, `feature/fine-strategy`, `test/unit-and-integration`, `test/selenium-e2e`, `docs/swagger-oper` `ci/github-actions`. Commit messages follow a short imperative summary with a scope prefix (e.g. `feat: add heavy-overdue fine strategy with boundary tests`).

[PLACEHOLDER: Append Git evidence produced on YOUR repository:] `git log --oneline --graph --all` screenshot (Appendix C), contribution of your own commits, and the README as delivered in this repository.

20 Pull-Request Review Evidence

The report requires at least one accepted PR and one rejected/revised PR with reviews. `docs/git-workflow.md` §3 contains the review checklist used (correctness, tests, readability, error handling, secrets, docs, scope).

[PLACEHOLDER: Paste real PR evidence: accepted PR link/screenshot with review comments; rejected or revised PR link/screenshot with the change requests and your follow-up commits (Appendix D). Do not fabricate reviews.]

21 Merge-Conflict Resolution Evidence

A safe, reproducible conflict exercise (two branches editing the same README line, merge, resolve markers, verify) is documented in `docs/git-workflow.md` §4.

[PLACEHOLDER: Include screenshots of the conflicted file with conflict markers and of the resolved commit (Appendix E).]

22 Continuous Integration (GitHub Actions)

`.github/workflows/ci.yml` runs on pushes and pull requests to `main` with three jobs: **backend-tests** (checkout, Node 20, `npm ci` from the lockfile, `npm run test:coverage` with the in-memory MongoDB — no private credentials required, coverage uploaded as an artifact), **frontend-build** (`npm ci` + `vite build`) and **selenium-e2e** (build + preview + browser test on the runner). Failures are diagnosed by opening the Actions run → failed job → step log.

[PLACEHOLDER: A green checkmark on GitHub only exists after you push and the runners succeed — include a screenshot of the successful run in Appendix F and state the run number honestly.]

23 Swagger/OpenAPI — Self-Directed Tool

What was learned and why. API documentation that stays usable after the developer leaves the room. **Application.** `server/docs/openapi.yaml` (OpenAPI 3.0) documents every implemented endpoint with request/response schemas and examples; `swagger-ui-express` serves interactive docs at `/api-docs`; the raw spec is exposed at `/api/openapi.json`. A live check on the development machine returned HTTP 200 for both (Appendix G).

Demonstration. Open `http://localhost:5000/api-docs`, expand `POST /api/books`, press *Try it out*, execute a sample book creation, and observe the 201 response — the same call the React form makes.

Benefits vs. alternatives. Compared with a README: structured, versionable, and rendered as interactive docs instead of prose that drifts. Compared with a Postman collection: shareable as a URL/spec, diff-able in Git, and vendor-neutral. **Limitations:** the spec is maintained by hand (no automatic generation from Express code), response schemas are summarised rather than exhaustive, and authentication schemes are not documented because the system has none.

Learning process (personalise before submission): resources used were the OpenAPI specification (`swagger.io/specification`) and the swagger-ui-express README; **[PLACEHOLDER: describe one difficulty you hit (e.g. YAML indentation) and how you resolved it]**.

24 Learning Log

The learning-log template with columns (date, task, resource, learning, challenge, solution, evidence, time) plus recommended official resources for React, Tailwind, Node/-Express, MongoDB/Mongoose, Jest, Supertest, Selenium, Git/GitHub, Actions, Ope-

nAPI and LaTeX, and a resource-evaluation method, is provided in `docs/learning-log.md`. Sample rows are explicitly marked **[EXAMPLE]** and must be replaced with your real sessions.

[PLACEHOLDER: Attach your completed learning log (Appendix H).]

25 Reflection Essay (500–700 words)

When I started this project, the goal was not to build the biggest library system in the world, but to walk through a complete software engineering process on a system small enough to understand end-to-end. The Smart Library & Fine Management System uses the MERN stack — React with Tailwind CSS on the front, Express and Mongoose with MongoDB behind a REST API — because it let me keep one language (JavaScript) across the whole project and focus on engineering practice rather than fighting tooling.

For the process model I chose the Incremental Model. The five modules — members, books, borrow/return, fines and audit — are naturally separable, so I could finish and test each slice before starting the next. This suited a single-developer sessional project: if something went wrong in the fine policy, member registration was already stable and demonstrable. [PERSONAL: describe one real problem you hit in a specific increment and how incremental delivery helped.]

Three design patterns did real work in the code. The **Singleton** `DatabaseManager` gives the process one shared connection manager, so the connection lifecycle has a single owner; I rejected Prototype because cloning a database manager would be meaningless — there is nothing useful to copy. The **Facade** `BorrowFacade` hides the multi-step issue/return workflow (member lookup, borrow limit, stock update, due date, fine calculation, audit logging) behind two readable methods; I rejected Adapter because no incompatible third-party interface needed translating. The **Strategy** pattern implements the fine policies — 10 BDT/day for students, zero for faculty, and a heavy-overdue decorator charging 20 BDT/day after the first seven days. I rejected State because the fine rules are interchangeable algorithms selected by role, not an object whose behaviour changes with internal state.

Test-Driven Development was applied most strictly to fine calculation. I recorded a real Red–Green cycle: the tests failed against the unimplemented strategy ("five overdue days $\rightarrow$ 50 BDT" expected 50, received 0), passed once the ten-taka-per-day rule was implemented, and the boundary cases (day $7 = 70$, day $8 = 90$) were locked in with further tests. Mocking mattered in the facade unit tests: repositories, the fine service and the audit service are all Jest mocks, so the tests verify coordination logic without a database and run in milliseconds. Integration tests with Supertest cover the real HTTP path against an isolated in-memory MongoDB, which catches wiring mistakes mocks cannot see. The Selenium test drives the actual React UI in Chrome — add book, add member, issue, return — and passed on my machine. [PERSONAL: describe one real

bug that Selenium or the integration tests found that unit tests missed.]

Continuous integration through GitHub Actions re-runs tests, coverage and the frontend build on every push, which turns "it works on my machine" into a shared, visible fact. As the self-directed tool I learned **Swagger/OpenAPI**: one YAML file now generates interactive documentation at `/api-docs`, and I can test any endpoint from the browser. I chose it over a README because a specification is structured, versionable and much harder to let drift out of date. [PERSONAL: mention which Swagger resources you actually used and one real difficulty, e.g. YAML indentation.]

The hardest part was [PERSONAL: replace with your real challenge — e.g. understanding atomic MongoDB updates, React select state in Selenium, or wiring dependency injection]. I resolved it by [PERSONAL: your real solution steps]. The learning resources I relied on were mostly official documentation (Mongoose, Jest, Express, Tailwind), which I evaluated by whether they matched the project's library versions and whether I could apply them within one sitting.

The system has clear limitations: no authentication, no payment flow, regex search instead of Atlas Search, and no multi-document transactions — the issue workflow instead uses an atomic conditional stock update, which I documented honestly rather than overclaiming. Next I want to learn proper authentication (JWT with role checks) and containerised deployment, because the realistic next step for this application is running it safely for real users rather than adding more features.

[FINAL STEP: after your viva rehearsals and real test runs, re-read the essay aloud, keep it between 500 and 700 words, and remove every bracketed marker.]

26 Limitations

1. No authentication/authorization; roles are labels only.
2. Fines are computed but not paid or waived.
3. In-memory demo mode (`start:memory`) is non-persistent by design.
4. Regex-based search instead of Atlas Search; no pagination.
5. No multi-document transactions; atomic conditional stock update is the documented alternative (safe at small scale, not a full transactional guarantee).
6. The OpenAPI spec is hand-maintained and can drift if endpoints change without updating the YAML.
7. CI status on GitHub must be evidenced by an actual successful run on your repository.

27 Future Improvements

JWT-based authentication with real role checks; fine payment records; pagination and Atlas Search; a replica-set deployment enabling true transactions; generating the OpenAPI spec from code (or tests asserting spec/route parity); Dockerised deployment; notification preferences; borrow-history export.

28 Conclusion

The Smart Library & Fine Management System delivers five working modules on the required MERN + Tailwind stack, applying the Incremental Model, three design patterns in genuine production paths (Singleton, Facade, Strategy), TDD with recorded Red–Green evidence, mocked unit tests, Supertest integration tests against an isolated database, a passing Selenium end-to-end workflow, a GitHub Actions pipeline and hand-authored Swagger documentation. The engineering artefacts — requirements, diagrams, tests, CI and documentation — describe the same system that the code implements, with limitations stated honestly rather than glossed over.

29 References

1. Express.js Team, "Express 4.x Guide," https://expressjs.com/en/guide/, accessed 2026.
2. MongoDB, Inc., "Mongoose ODM Documentation," https://mongoosejs.com/docs/, accessed 2026.
3. MongoDB, Inc., "MongoDB Manual: Documents and Schema Design," https://www.mongodb.com/docs/manual/, accessed 2026.
4. React Team, "React Documentation," https://react.dev/, accessed 2026.
5. Tailwind Labs, "Tailwind CSS v3 Documentation," https://v3.tailwindcss.com/docs, accessed 2026.
6. Jest Team, "Jest Testing Framework," https://jestjs.io/docs/, accessed 2026.
7. Supertest contributors, "supertest," https://github.com/ladjs/supertest, accessed 2026.
8. Selenium Project, "Selenium WebDriver Documentation," https://www.selenium.dev/documentation/webdriver/, accessed 2026.
9. OpenAPI Initiative, "OpenAPI Specification 3.0.3," https://swagger.io/specification/, accessed 2026.

10. GitHub, "GitHub Actions Documentation," https://docs.github.com/en/actions, accessed 2026.

11. Gamma, E., Helm, R., Johnson, R., Vlissides, J. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1995.

12. Sommerville, I. *Software Engineering*, 10th ed. Pearson, 2016.

13. mongodb-memory-server contributors, https://github.com/typegoose/mongodb-memory-server, accessed 2026.

A Key Code Excerpts

Full source is in the project repository. Additional excerpts: the complete Singleton (`server/src/database/DatabaseManager.js`), the heavy-overdue decorator (`server/src/strategi` and the atomic stock decrement (`server/src/repositories/BookRepository.js`).

Listing 3: Heavy-overdue decorator (core)

```
calculateFine(overdueDays) {
  const days = Number(overdueDays);
  if (!Number.isInteger(days) || days < 0) {
    throw new Error('overdueDays must be a non-negative whole number
        ');
  }
  if (days <= this.threshold) {
    return this.baseStrategy.calculateFine(days);
  }
  const basePart = this.baseStrategy.calculateFine(this.threshold);
  const extraDays = days - this.threshold;
  return basePart + extraDays * this.heavyRate;
}
```

Listing 4: Atomic availability decrement

```
async decrementAvailable(id) {
  return this.model.findOneAndUpdate(
    { _id: id, availableCopies: { $gt: 0 } },
    { $inc: { availableCopies: -1 } },
    { new: true }
  );
}
```

B Test Results

Unit + integration run (recorded): 4 suites, 64 tests, all passing (`npm run test`). **[PLACEHOLDER: re-run at submission and paste your terminal output / screenshot.]**

B.1 Coverage Summary

Recorded with `npm run test:coverage` on the development machine:

Table 4: Jest coverage (actual recorded run)

File	% Stmts	% Branch	% Funcs	% Lines
All files	89.77	73.33	91.01	90.88
src/database/DatabaseManager.js	82.60	80.00	80.00	82.60
src/facades/BorrowFacade.js	95.08	82.85	100	95.08
src/middleware/errorHandler.js	35.29	7.14	75.00	33.33
src/middleware/validate.js	92.00	90.90	100	91.66
src/models/*	96.66	50.00	100	96.66
src/repositories/*	94.64	70.58	95.45	96.22
src/services/FineService.js	95.65	90.90	100	95.45
src/services/AuditService.js	85.71	33.33	100	85.71
src/strategies/*	93.75	90.47	83.33	93.75
src/utils/*	100	100	100	100

Interpretation: statement/line coverage is high for business logic; the main gap is untested branches of the centralised error handler (e.g. specific Mongoose error shapes). Identified improvement: add unit tests that feed `ValidationError`, `CastError` and duplicate-key errors into `errorHandler`.

B.2 TDD Red–Green Record

Red run (unimplemented strategy): `Tests: 2 failed, 16 skipped, 3 passed, 21 total` — failures: "one overdue day → 10 BDT" and "five overdue days → 50 BDT". Green run (implemented): `Tests: 16 skipped, 5 passed, 21 total`. Full logs: `docs/evidence/tdd-red-run.txt`, `tdd-green-run.txt`.

B.3 Selenium E2E

Recorded output (development machine):

Listing 5: e2e run (recorded)

```
STEP 1 OK - dashboard loaded
STEP 2 OK - book added, toast shown: Book "Selenium Test Book ..."
   added to the catalog.
STEP 3 OK - member registered and visible in table
STEP 4 OK - book issued, toast shown: Book issued to E2E Member. Due
   date: 2026-11-09
STEP 5 OK - active borrow row visible
STEP 6 OK - book returned, toast shown: Book returned on time. No
   fine.
```

```
E2E TEST PASSED - issue/return workflow works through the UI
```

[PLACEHOLDER: Add a screenshot of your own run and of the headed browser.]

C Git Evidence

[PLACEHOLDER: Screenshots: git log graph, your commits, branch list.]

D Pull-Request Evidence

[PLACEHOLDER: Accepted PR with reviews; rejected/revised PR with comments.]

E Merge-Conflict Evidence

[PLACEHOLDER: Conflict markers screenshot + resolved commit.]

F CI Evidence

[PLACEHOLDER: GitHub Actions successful run screenshot (run number + date).]

G Swagger Evidence

[PLACEHOLDER: Screenshot of /api-docs with a "Try it out" response.] Live checks performed during development: `GET /api-docs` → `200`; `GET /api-openapi.json` returned OpenAPI 3.0.3 with all documented paths.

H Learning Log

[PLACEHOLDER: Your completed learning log (see docs/learning-log.md).]

I Application Screenshots

[PLACEHOLDER: Dashboard, Members, Books, Issue/Return, Fines, Audit pages from your running application (add PNGs to figures/ and reference them here).]